

Null Operators na L11

NULLABLE TYPE, NULL SAFETY, NULL COALESCING, NULL
ASSERTION, NULL-AWARE ASSIGNMENT & TERNARY OPERATOR

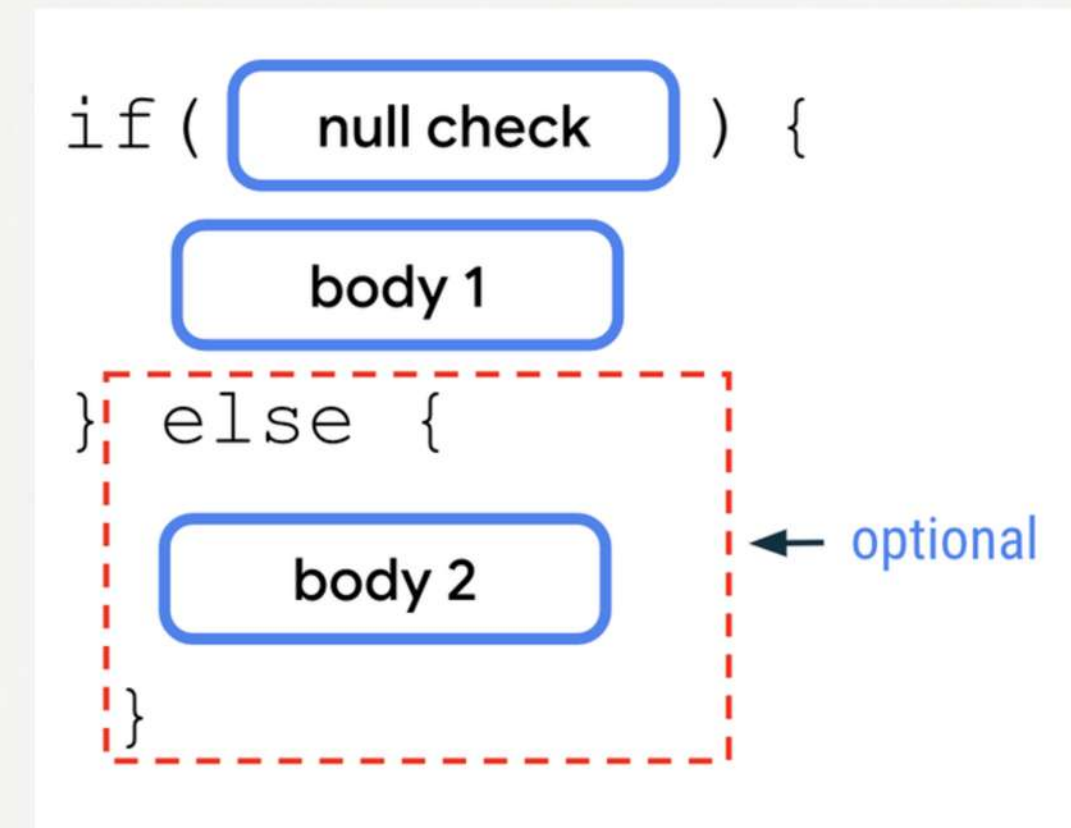
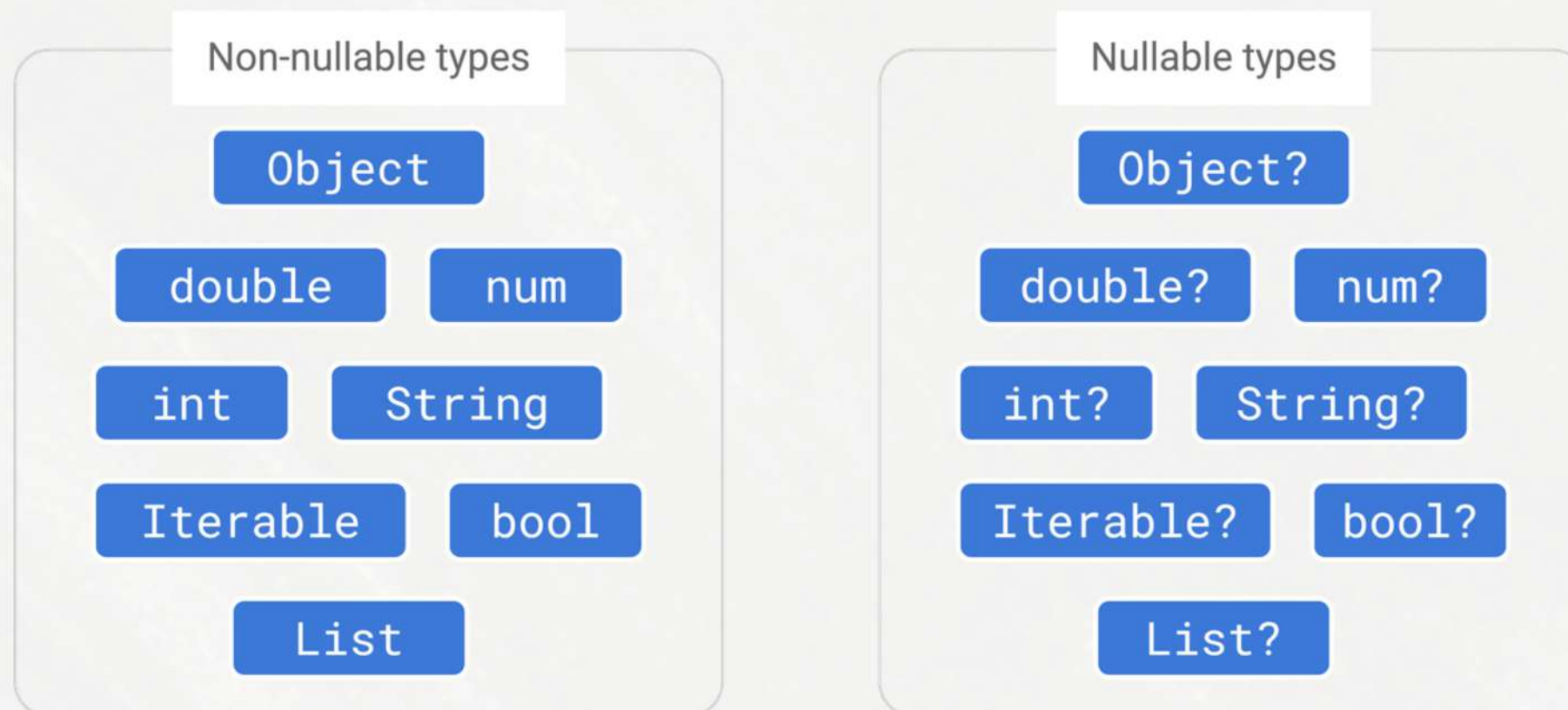
Nathalia Barbosa
NFAB

Thales Bezerra
TOB2

Visão Geral

Expansão da L1 com valor nulo, null safety, operadores de null e operador ternário.

- **Inspirações:** Dart, Kotlin, C e outras linguagens que possuem esses operadores.



BNF Completa

Programa ::= Comando

Comando ::= Atribuicao

| **AtribuicaoSeNulo**
| ComandoDeclaracao
| While
| IfThenElse
| IO
| Comando ";" Comando
| Skip

Skip ::=

Atribuicao ::= Id "==" Expressao

AtribuicaoSeNulo := Id "?:" Expressao

Expressao ::= Valor

| ExpUnaria
| ExpBinaria
| **ExpTernaria**
| Id

Valor ::= ValorConcreto

ValorConcreto ::= ValorInteiro

| ValorBooleano
| ValorString
| **ValorNulo**

ExpUnaria ::= "-" Expressao

| "not" Expressao
| "length" Expressao
| **Expressao "!"**

ExpBinaria ::= Expressao "+" Expressao

| Expressao "-" Expressao
| Expressao "and" Expressao
| Expressao "or" Expressao
| Expressao "==" Expressao
| Expressao "++" Expressao
| **Expressao "??" Expressao**

**ExpTernaria ::= Expressao "?" Expressao ":"
Expressao**

ComandoDeclaracao ::= "{" Declaracao ";"
Comando "}"

Declaracao ::= DeclaracaoVariavel
| DeclaracaoComposta

DeclaracaoVariavel ::= "var" Id "=" Expressao
| **DeclaracaoOptional**

**DeclaracaoOptional ::= "var" "optional" Id "="
Expressao**

DeclaracaoComposta ::= Declaracao ","
Declaracao

While ::= "while" Expressao "do" Comando

IfThenElse ::= "if" Expressao "then" Comando
"else" Comando

IO ::= "write" "(" Expressao ")"
| "read" "(" Id ")"

Nullable Type

Variáveis podem ser (re)assinaladas para o valor Null, desde que tenham a declaração com keyword "optional".

```
1 + inputs:
2 + expect: 5 null
3 +
4 + {
5 +   var optional a = 5,
6 +   var optional b = null;
7 +   write(a);
8 +   a := null;
9 +   b := a;
10 +   write(b)
11 + }
```

```
1 + inputs:
2 + expect: Erro de tipo
3 +
4 + {
5 +   var x = null;
6 +   write(x)
7 + }
```


Nullable Type - alterações

- Valor Null (*ValorNulo*);
- Keyword "optional", que determina se um valor pode ser assinalado para Null ou não (*TipoOptional*);
- *DeclaracaoOptional*;
- *checaTipo* de *Atribuicao* e *DeclaracaoVariavel* → verifica também se a expressão é Null ou Opcional, lançando erro caso necessário.

Novidades na BNF:

```
ValorConcreto ::= ValorInteiro  
                | ValorBooleano  
                | ValorString  
                | ValorNulo
```

```
DeclaracaoVariavel ::= "var" Id "=" Expressao  
                    | DeclaracaoOptional
```

```
DeclaracaoOptional ::= "var" "optional" Id "="  
Expressao
```

Nullable Type - interpretador

Novidades no interpretador:

- Modificar declaração variável para possivelmente ser uma declaração opcional
- Definir *isOptional* para *true* caso a keyword tenha sido utilizada

```
694 + Declaracao PDeclaracaoVariavel() :
695     {
696         Id id;
697         Expressao exp;
698 +     Declaracao retorno;
699 +     boolean isOptional = false;
700     }
701     {
702 +     <VAR> ( <OPTIONAL> { isOptional = true; } )? id = PId()
703 +     <ASSIGN> exp = PExpressao()
704 +     { if (isOptional) retorno = new DeclaracaoOptional(id,
705 +     exp); else retorno = new DeclaracaoVariavel(id, exp); }
706     {
707         return retorno;
708     }
```


Null Safety

Caso uma operação tenha risco de causar erro de execução por causa do Null, ela lança um erro de compilação em vez disso.

```
1  + inputs:
2  + expect: 5
3  +
4  + { var optional y = 2 ;
5  +   if (y == null) then
6  +     Skip
7  +   else
8  +     { var x = y + 3 ;
9  +       write(x)
10 +     }
11 + }
```

```
1  + inputs:
2  + expect: Erro de tipo
3  +
4  + { var optional y = 2 ;
5  +   { var x = y + 3 ;
6  +     write(x)
7  +   }
8  + }
```

Null Safety – alterações

- Alterações no **checa_tipo** dos operadores **if** e **ternário**:
 - Verifica ramos **then** e **else** para ver se regras do null safety estão sendo violadas;
 - Faz o mesmo para os dois lados do ternário.
- *TipoOptional* agora tem a flag *isRefinado* que indica se ele foi considerado “safe”

Não houve alterações na BNF

Null Coalescing

Operador binário que retorna o lado direito da operação caso o operador seja Null, ou o esquerdo caso não seja Null. Atua como um valor "default".

```
1 + expect: 2
2 +
3 + {
4 +   var y = 2,
5 +   var x = y ?? 5;
6 +   write(x)
7 + }
```

```
1 + expect: 5
2 +
3 + {
4 +   var optional y = null,
5 +   var x = y ?? 5;
6 +   write(x)
7 + }
```

Null Coalescing - alterações

- **ExpNullCoalescing:** implementação da expressão “??”.
 - avalia o valor da esquerda. se for nulo, avalia o da direita.
 - se o da esquerda for nulo, verifica validade do da direita.
 - se não for nulo, é válido se o tipo da esquerda for igual ao tipo da direita.

Novidades na BNF:

```
ExpBinaria ::= Expressao “+” Expressao
            | Expressao “-” Expressao
            | Expressao “and” Expressao
            | Expressao “or” Expressao
            | Expressao “==” Expressao
            | Expressao “++” Expressao
            | Expressao “??” Expressao
```


Null Coalescing - interpretador

Novidades no interpretador:

- Usa lookahead para checar pelo padrão em *PExpressao*

```
652 Expressao PExpressao() :
653 {
654     Expressao retorno;
655 }
656 {
657     (
658         LOOKAHEAD (PExpBase() <HOOK>)
659         retorno = PExpTernaria()
660 + | LOOKAHEAD (PExpBase() <NULL_COALESCING>)
661 +         retorno = PExpNullCoalescing()
```

```
670 + ExpNullCoalescing PExpNullCoalescing() :
671 + {
672 +     Expressao esq;
673 +     Expressao dir;
674 + }
675 + {
676 +     esq = PExpBase()
677 +     <NULL_COALESCING>
678 +     dir = PExpressao()
679 +     {return new ExpNullCoalescing(esq, dir);}
680 + }
681 +
```

Operador Ternário

Operador "? :" que atua de forma semelhante à um **if-then-else**, mas para atribuir valores

```
1 + expect: 1 10 null
2 +
3 + {
4 +   var optional x = (4 == 3) ? null : 1,
5 +   var optional y = (x == null) ? null : 10,
6 +   var optional z = (4 == 4) ? null : 1;
7 +   write(x);
8 +   write(y);
9 +   write(z)
10 + }
```

```
1 + expect: Erro de tipo
2 +
3 + {
4 +   var x = 4 == 3 ? null : 1;
5 +   write(x)
6 + }
```


Operador Ternário - alterações

- Novo tipo de expressão: *ExpTernaria*;
- Implementação do ternário em *ExpOpTernario*;
- Alteração em *DeclaracaoOpcional* para suportar ternários.

Novidades na BNF:

```
Expressao ::= Valor  
           | ExpUnaria  
           | ExpBinaria  
           | ExpTernaria  
           | Id
```

```
ExpTernaria ::= Expressao "?" Expressao ":"  
Expressao
```

Operador Ternário - interpretador

Novidades no interpretador:

- Implementa lookahead em *PExpressao* para checar por comandos ternários

```
632 Expressao PExpressao() :  
633 {  
634     Expressao retorno;  
635 }  
636 {  
637     (  
638 +     LOOKAHEAD (PExpPrimaria() <HOOK>)  
639 +     retorno = PExpTernaria()  
640 +     |
```

```
542 + Expressao PExpTernaria() :  
543 + {  
544 +     Expressao esq;  
545 +     Expressao mei;  
546 +     Expressao dir;  
547 + }  
548 + {  
549 +     esq = PExpPrimaria()  
550 +     <HOOK>  
551 +     mei = PExpressao()  
552 +     <COLON>  
553 +     dir = PExpressao()  
554 +     {return new ExpOpTernario(esq, mei, dir);}  
555 + }  
556 +
```


Null Assertion

Colocando a keyword "!" após acessar a variável, garantimos ao compilador que o valor dela não é nulo, essencialmente permitindo ignorar o Null safety.

```
1 + inputs:
2 + expect: 3
3 +
4 + {
5 +     var optional y = 1 ;
6 +     {
7 +         var x = y! + 2 ;
8 +         write(x)
9 +     }
10 + }
```

```
1 + inputs:
2 + expect: Erro de execucao: valor nulo encontrado na assercao (!).
3 +
4 + {
5 +     var optional y = null ;
6 +     {
7 +         var x = y! + 2 ;
8 +         write(x)
9 +     }
10 + }
```

Null Assertion – alterações

- Nova expressão: *ExpNullAssertion*, que implementa o operador “!”
- Novo tipo: *TipoCuringa*, que permite pular as verificações de tipo. Ele é retornado se o valor do elemento for nulo, impedindo que dê um erro de compilação.

Novidades na BNF:

```
ExpUnaria ::= “-” Expressao  
           | “not” Expressao  
           | “length” Expressao  
           | Expressao “!”
```


Null Assertion - interpretador

Novidades no interpretador:

- Modifica *PExpPrimaria* para identificar o sinal de “!” (<BANG>)

```
Expressao PExpPrimaria() :
{
    Expressao retorno;
}
{
    (
        retorno = PId()
    | retorno = PValor()
    | <LPAREN> retorno = PExpressao() <RPAREN>
    )
+      (
+          <BANG> { retorno = new ExpNullAssertion(retorno); }
+      )*
+      {
+          return retorno;
+      }
}
```

Null-Aware Assignment

Operador binário que atribui um valor ao lado esquerdo se, e somente se, o operando do lado esquerdo for nulo. Caso contrário, mantém o valor.

```
1 + expect: 3
2 +
3 + {
4 +   var optional y = 3;
5 +   y ?:= 5;
6 +   y ?:= 10;
7 +   write(y)
8 + }
```

```
1 + expect: 5
2 +
3 + {
4 +   var optional y = null;
5 +   y ?:= 5;
6 +   y ?:= 10;
7 +   write(y)
8 + }
```

Null-Aware Assignment - alterações

- **AtribuicaoSeNulo:** verifica se o valor do id é nulo. Se for, usa o changeValor com a avaliação da expressão. Depois, retorna o ambiente.
 - Se tipo da exp for nulo e o id não for opcional, retorna falso no checaTipo.
 - Se tipo da exp não for nulo, checa se tipo do id é igual ao da expressão.

Novidades na BNF:

```
Comando ::= Atribuicao
          | AtribuicaoSeNulo
          | ComandoDeclaracao
          | While
          | IfThenElse
          | IO
          | Comando ";" Comando
          | Skip
```

```
AtribuicaoSeNulo := Id "?:=" Expressao
```


Null-Aware Assignment – interpretador

Novidades no interpretador:

- Implementa lookahead em *PComandoSimples* para checar se a atribuição se encaixa no se-nulo

```
684 Comando PComandoSimples() :  
  
688 {  
689     (  
690         retorno = PSkip()  
691 +     | LOOKAHEAD(PId() <NULL_AWARE_ASSIGN>) retorno =  
        PAtribuicaoSeNulo()  
692         | retorno = PAtribuicao()  
693         | retorno = PComandoDeclaracao()  
694         | retorno = PWhile()  
  
701     }  
702 }  
703  
704 + AtribuicaoSeNulo PAtribuicaoSeNulo() :  
705 + {  
706 +     Id id;  
707 +     Expressao exp;  
708 + }  
709 + {  
710 +     id = PId() <NULL_AWARE_ASSIGN> exp = PExpressao()  
711 +     { return new AtribuicaoSeNulo(id, exp); }  
712 + }  
713 +
```


Script para testes

Para verificar o comportamento desejado de cada uma das operações implementadas, fizemos alguns testes que executam por meio de um script.

```
Testes
├── GENERAL_bad_syntax.txt
├── GENERAL_good_syntax.txt
├── NULL_ASSERTION_error.txt
├── NULL_ASSERTION_good.txt
├── NULL_AWARE_ASSIGN_variable_different_types.txt
├── NULL_AWARE_ASSIGN_variable_with_value.txt
├── NULL_AWARE_ASSIGN_variable_without_value.txt
├── NULL_COALESCING_variable_different_types.txt
├── NULL_COALESCING_variable_with_value_non_optional.txt
├── NULL_COALESCING_variable_with_value_optional.txt
├── NULL_COALESCING_variable_without_value.txt
├── NULL_COALESCING_variable_without_value_non_optional.txt
└── ...

=== Iniciando Testes do Compilador ===

Compilando projeto... WARNING: A terminally deprecated method in sun.misc.Unsafe has been called by com.google.common.util.concurrent.PerfCounter (file:/usr/share/maven/lib/guava.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.PerfCounter
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.google.common.util.concurrent.PerfCounter (file:/usr/share/maven/lib/guava.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.PerfCounter
icHelper
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.google.common.util.concurrent.PerfCounter (file:/usr/share/maven/lib/guava.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.PerfCounter
Java Compiler Compiler Version 5.0 (Parser)
(type "javacc" with no arguments for help)
Reading from file /workspace/Imperativa/Compiler/TokenMgrError.java
File "TokenMgrError.java" does not exist.
File "ParseException.java" does not exist.
File "Token.java" does not exist.
Will attempt to generate these files.
File "JavaCharStream.java" does not exist.
Will attempt to generate this file.
Parser generated successfully.
[OK]

▶ Teste: GENERAL_bad_syntax.txt
-----
Status: [OK]
Resultado Esperado Alcançado: Encountered errors
=====

▶ Teste: GENERAL_good_syntax.txt
-----
Status: [OK]
Resultado Esperado Alcançado: 3 1 2 7 true tchau
=====

▶ Teste: OPTIONAL_reassign_nullable.txt
-----
Status: [OK]
Resultado Esperado Alcançado: 5 null
=====

▶ Teste: OPTIONAL_type_error.txt
-----
Status: [OK]
Resultado Esperado Alcançado: Erro de tipo
=====

▶ Teste: WRITE_NULL.txt
-----
Status: [OK]
Resultado Esperado Alcançado: null
=====

Resumo: Passou 28 / Falhou 0
```

Obrigado!

[HTTPS://GITHUB.COM/NATHALIAFAB/NULLOPERATORSLI1](https://github.com/nathaliafab/nulloperatorsli1)